

# Spring Boot for Beginners

A beginner-friendly introduction for new joiners — understand what Spring Boot is, why it matters, and how to get started in enterprise Java development.

 INDUCTION TRAINING

 JAVA BACKEND

 ENTRY LEVEL

**[naresh.chaurasia@cognizant.com](mailto:naresh.chaurasia@cognizant.com)**

# Spring vs Spring Boot

Before diving in, let's clear up the most common question freshers ask: **what's the difference between Spring and Spring Boot?**

| Area             | Spring Framework                   | Spring Boot                                    |
|------------------|------------------------------------|------------------------------------------------|
| Configuration    | More manual configuration required | Auto-configuration out of the box              |
| Boilerplate Code | More setup code needed             | Much less setup — sensible defaults            |
| Server           | External deployment often needed   | Embedded server included (Tomcat)              |
| Purpose          | Full framework foundation          | Rapid application development on top of Spring |

✔ **Key Takeaway:** Spring Boot is built on Spring and makes Spring much faster and easier to use.

# Why Do We Need Spring?

Before Spring, building Java enterprise applications was painful. Here's what developers had to deal with every day:

## Too Much Configuration

Developers spent more time configuring than coding actual business logic.

## Tight Coupling

Classes were tightly bound together, making changes risky and difficult.

## Hard to Test

Tightly coupled code was nearly impossible to unit test in isolation.

## Slower Development

Repetitive boilerplate slowed teams down and delayed delivery.

 Spring was created to solve exactly these problems — giving Java developers a cleaner, more productive way to build applications.

# What is Spring?

Spring is a powerful Java framework that helps you build enterprise-grade applications — without the mess of traditional Java EE. Think of it as a helpful toolkit that handles the boring stuff so you can focus on writing real features.

## Three Big Ideas in Spring

These concepts sound fancy, but they're actually quite simple once you see them in action.



### IoC — Inversion of Control

Instead of your code creating objects, Spring creates and manages them for you. You just ask for what you need.



### Dependency Injection

Spring "injects" the objects your class needs automatically — like a waiter bringing your order without you going to the kitchen.



### AOP — Aspect-Oriented Programming

Add cross-cutting concerns like logging or security in one place, without touching every class. Write it once, apply it everywhere.

# Why Was Spring Boot Created?

Even with Spring, setting up a new project was still a headache. Here's what developers faced before Spring Boot:

## 1 Mountains of XML Configuration

Every project needed lengthy XML files just to wire things together — before writing a single line of business logic.

## 2 Repeated Setup Work

Every new project meant copying the same boilerplate setup. Developers wasted hours on configuration that never changed.

## 3 Slow Onboarding for New Developers

Freshers joining a project had to understand complex configuration before they could contribute anything meaningful.

❏ **The Solution:** Spring Boot was created to eliminate all of this friction — so developers (including you!) can go from zero to a running application in minutes, not days.

# What is Spring Boot?

Spring Boot is an extension of the Spring Framework that lets you create **stand-alone, production-ready applications quickly** — with minimal fuss.



## Opinionated Defaults

Spring Boot makes sensible choices for you. You can always override them, but you rarely need to.



## Less Configuration

No more XML files. Spring Boot auto-configures your application based on what's on the classpath.



## Faster Development

Start coding your features immediately. Spring Boot handles the infrastructure setup for you.



## Easier Setup

One JAR file, one command. Your entire application — including the server — is bundled and ready to run.

# Key Features of Spring Boot

Here are the five features you'll hear about most often on real enterprise projects:

1

## Auto-Configuration

Spring Boot detects what libraries you're using and configures them automatically — no manual wiring needed.

2

## Starter Dependencies

Pre-packaged dependency bundles (e.g. `spring-boot-starter-web`) so you add one line and get everything you need.

3

## Embedded Server

Tomcat is built right in. Run your app with `java -jar app.jar` — no separate server installation required.

4

## Externalised Configuration

Manage settings via `application.properties` or environment variables — easy to change without recompiling.

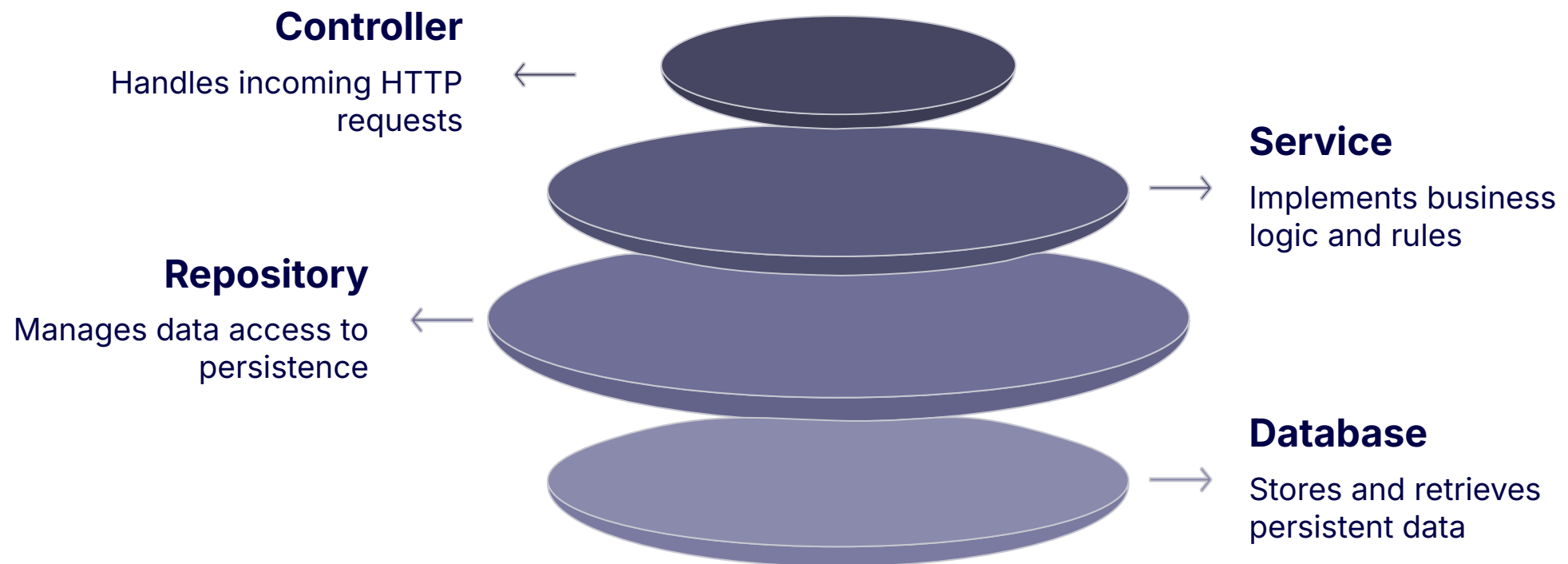
5

## Production-Ready Features

Built-in health checks, metrics, and monitoring via Spring Actuator — your app is enterprise-ready from day one.

# Structure of a Spring Boot Application

Every Spring Boot application follows a clean, layered architecture. Each layer has one clear responsibility — this makes the code easy to understand, test, and maintain.



## **Controller**

Receives HTTP requests and sends back responses. The entry point for your API.

## **Service**

Contains the business logic. Decides what to do with the data.

## **Repository**

Handles all database operations — saving, fetching, updating, deleting.

## **Database**

Stores the actual data. Spring Boot works with MySQL, PostgreSQL, H2, and more.

 **Tip for freshers:** When you join a project, look for these four layers first — they'll help you navigate any Spring Boot codebase quickly.

# Where is Spring Boot Used in Real Projects?

Spring Boot isn't just a learning exercise — it powers real systems at companies around the world, including right here. Here's where you'll encounter it:



## REST APIs

The most common use case. Spring Boot makes building and exposing APIs fast and straightforward.



## Microservices

Large enterprise systems are broken into small, independent services — Spring Boot is the go-to choice for each one.



## Enterprise Applications

Internal tools, customer portals, and business platforms — Spring Boot handles the backend reliably at scale.



## Cloud-Native Systems

Spring Boot integrates seamlessly with AWS, Azure, and GCP — ideal for modern cloud deployments.

📌 On most enterprise Java projects you join here, you will be working with Spring Boot from day one. This is not just theory — it's your daily toolkit.

# Key Takeaways for Freshers

Here's everything you need to remember from today's session — keep this as your quick reference as you start your journey.

01

## Spring is the Foundation

Spring Framework provides the core building blocks — IoC, Dependency Injection, and AOP — that power modern Java applications.

02

## Spring Boot Makes it Easier and Faster

Spring Boot removes the complexity of Spring setup with auto-configuration, embedded servers, and starter dependencies.

03

## Widely Used in Enterprise Java

From REST APIs to microservices and cloud platforms — Spring Boot is the industry standard for backend Java development.

04

## A Strong Skill for Your Career

Whether you're aiming for backend or full-stack development, Spring Boot is one of the most in-demand skills in enterprise Java.

*"Learning Spring Boot is one of the fastest ways to become productive in modern Java projects."*

✅ **Next Steps:** Set up Spring Initializr, create your first project, and build a simple REST API. You've got this! 🚀